Experiment 11

Fruit Ninja: Python Mini Project

Aim: Design and implement a mini project using Python concepts covered in the course.

File: app.py

```
import sys
import os
import pygame

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from helper.coords import SCREEN_WIDTH, SCREEN_HEIGHT, FPS
from helper.gui import Graphics
from helper.handler import GameLogic

def main():
    pygame.init()
    pygame.display.set_caption("Fruit Ninja - Python Mini Project")
    screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
    clock = pygame.time.Clock()

    graphics = Graphics()
    game = GameLogic(graphics)

    running = True
    while running:
        clock.tick(FPS)
        running = game.handle_input()
        game.update()
        game.draw(screen)

    pygame.quit()
    sys.exit()

if __name__ == "__main__":
    main()
```

File: requirements.txt

```
pygame>=2.5.0
Pillow>=10.0.0
```

File: helper/__init__.py

```
from helper.coords import screen_width, screen_height, fps, gravity, get_paths
from helper.physics import Fruit, HalfPiece, spawn_item
from helper.gui import Graphics
from helper.handler import GameLogic
```

File: helper/coords.py

```
import os

SCREEN_WIDTH = 800
```

```
SCREEN_HEIGHT = 600
FPS = 60
GRAVITY = 0.22
FRUIT_SIZE = 70

screen_width = SCREEN_WIDTH
screen_height = SCREEN_HEIGHT
fps = FPS
gravity = GRAVITY

def get_asset_paths():
    base_dir = os.path.dirname(os.path.abspath(__file__))
    candidates = [
        os.path.join(base_dir, "../Material"),
        os.path.join(base_dir, "../../Material"),
        os.path.join(os.getcwd(), "Material"),
        os.path.join(os.getcwd(), "../Material"),
    ]
    mat_dir = os.path.abspath(os.path.join(base_dir, "../Material"))
    for c in candidates:
        if os.path.isdir(c):
            mat_dir = os.path.abspath(c)
            break

    return {
        "apple": os.path.join(mat_dir, "apple.png"),
        "mango": os.path.join(mat_dir, "mango.png"),
        "strawberry": os.path.join(mat_dir, "strawberry.png"),
        "bomb": os.path.join(mat_dir, "bomb.png"),
        "background": os.path.join(mat_dir, "plank.jpg")
    }

get_paths = get_asset_paths
```

File: helper/gui.py

```
import os
import pygame
from helper.coords import SCREEN_WIDTH, SCREEN_HEIGHT, FRUIT_SIZE, get_asset_paths

class Graphics:
    def __init__(self, size=FRUIT_SIZE):
        self.size = size
        self.paths = get_asset_paths()
        self.images = {}
        self.halves = {}
        self.background = None

        try:
            self.font_large = pygame.font.SysFont("Arial", 42, bold=True)
            self.font_medium = pygame.font.SysFont("Arial", 26, bold=True)
        except Exception:
            self.font_large = pygame.font.Font(None, 48)
            self.font_medium = pygame.font.Font(None, 30)

        self._load_textures()

    def _load_textures(self):
        bg_path = self.paths.get("background")
        if bg_path and os.path.exists(bg_path):
```

```
        try:
            bg = pygame.image.load(bg_path).convert()
            self.background = pygame.transform.scale(bg, (SCREEN_WIDTH, SCREEN_HEIGHT))
        except Exception:
            self.background = self._make_fallback_bg()
    else:
        self.background = self._make_fallback_bg()

    for name in ["apple", "mango", "strawberry", "bomb"]:
        path = self.paths.get(name)
        if path and os.path.exists(path):
            try:
                img = pygame.image.load(path).convert_alpha()
                scaled = pygame.transform.scale(img, (self.size, self.size))
                self.images[name] = scaled
                self._create_halves(name, scaled)
            except Exception:
                self._make_fallback_fruit(name)
        else:
            self._make_fallback_fruit(name)

def _make_fallback_bg(self):
    surf = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT))
    surf.fill((60, 40, 25))
    return surf

def _make_fallback_fruit(self, name):
    surf = pygame.Surface((self.size, self.size), pygame.SRCALPHA)
    colors = {
        "apple": (220, 30, 30),
        "mango": (240, 160, 20),
        "strawberry": (230, 20, 80),
        "bomb": (30, 30, 30)
    }
    col = colors.get(name, (200, 200, 200))
    pygame.draw.circle(surf, col, (self.size // 2, self.size // 2), self.size // 2 - 2)
    self.images[name] = surf
    self._create_halves(name, surf)

def _create_halves(self, name, img):
    w, h = img.get_size()
    half_w = w // 2

    left_surf = img.subsurface(pygame.Rect(0, 0, half_w, h)).copy()
    right_surf = img.subsurface(pygame.Rect(half_w, 0, w - half_w, h)).copy()

    self.halves[name] = (left_surf, right_surf)

def draw_game(self, screen, fruits, halves, score, lives, blade_trail, is_mouse_down, game_over):
    screen.blit(self.background, (0, 0))

    for h in halves:
        pair = self.halves.get(h.kind)
        if pair:
            tex = pair[0] if h.is_left else pair[1]
            screen.blit(tex, (int(h.x), int(h.y)))

    for f in fruits:
        if not f.sliced:
            tex = self.images.get(f.kind)
            if tex:
```

```
            screen.blit(tex, (int(f.x), int(f.y)))

    if is_mouse_down and len(blade_trail) > 1:
        for i in range(len(blade_trail) - 1):
            pygame.draw.line(screen, (255, 255, 255), blade_trail[i], blade_trail[i + 1], 3)

    score_text = self.font_medium.render(f"Score: {score}", True, (255, 255, 255))
    screen.blit(score_text, (20, 20))

    lives_str = "Lives: " + "X " * max(0, lives)
    lives_text = self.font_medium.render(lives_str.strip(), True, (255, 80, 80))
    screen.blit(lives_text, (SCREEN_WIDTH - lives_text.get_width() - 20, 20))

    if game_over:
        overlay = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 160))
        screen.blit(overlay, (0, 0))

        go_txt = self.font_large.render("GAME OVER", True, (255, 50, 50))
        go_rect = go_txt.get_rect(center=(SCREEN_WIDTH // 2, SCREEN_HEIGHT // 2 - 30))
        screen.blit(go_txt, go_rect)

        res_txt = self.font_medium.render("Click or Press SPACE to Restart", True, (255, 255, 255))
        res_rect = res_txt.get_rect(center=(SCREEN_WIDTH // 2, SCREEN_HEIGHT // 2 + 30))
        screen.blit(res_txt, res_rect)
```

File: helper/handler.py

```
import random
import pygame
from helper.physics import HalfPiece, spawn_item

class GameLogic:
    def __init__(self, graphics):
        self.graphics = graphics
        self.score = 0
        self.lives = 3
        self.game_over = False

        self.fruits = []
        self.halves = []

        self.blade_trail = []
        self.is_mouse_down = False
        self.last_pos = None

        self.spawn_timer = 0
        self.spawn_interval = 90

    def reset(self):
        self.score = 0
        self.lives = 3
        self.game_over = False
        self.fruits.clear()
        self.halves.clear()
        self.blade_trail.clear()
        self.spawn_timer = 0

    def handle_input(self):
        for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
            return False

        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_q:
                return False
            elif event.key == pygame.K_SPACE or event.key == pygame.K_r:
                if self.game_over:
                    self.reset()

        elif event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 1:
                self.is_mouse_down = True
                pos = event.pos
                self.blade_trail = [pos]
                self.last_pos = pos

                if self.game_over:
                    self.reset()
                else:
                    self._check_slice(pos)

        elif event.type == pygame.MOUSEBUTTONUP:
            if event.button == 1:
                self.is_mouse_down = False
                self.blade_trail.clear()
                self.last_pos = None

        elif event.type == pygame.MOUSEMOTION:
            pos = event.pos
            if self.is_mouse_down and not self.game_over:
                self.blade_trail.append(pos)
                if len(self.blade_trail) > 8:
                    self.blade_trail.pop(0)

                if self.last_pos:
                    mid = ((self.last_pos[0] + pos[0]) / 2, (self.last_pos[1] + pos[1]) / 2)
                    self._check_slice(mid)

                self._check_slice(pos)
                self.last_pos = pos

    return True

def _check_slice(self, pos):
    for f in self.fruits:
        if not f.sliced and f.is_hit(pos):
            self._slice(f)

def _slice(self, fruit):
    fruit.sliced = True

    if fruit.kind == "bomb":
        self.lives -= 1
        if self.lives <= 0:
            self.game_over = True
    else:
        self.score += 1
        self.halves.append(HalfPiece(fruit.kind, True, fruit.x - 5, fruit.y, fruit.vx - 2.0, fruit.vy - 1.0, fruit.size))
        self.halves.append(HalfPiece(fruit.kind, False, fruit.x + 5, fruit.y, fruit.vx + 2.0, fruit.vy - 1.0, fruit.size))
```

```
    def update(self):
        if self.game_over:
            return

        self.spawn_timer += 1
        if self.spawn_timer >= self.spawn_interval:
            self.spawn_timer = 0
            count = random.choice([1, 2])
            bomb_chance = min(0.3, 0.15 + self.score * 0.01)
            for _ in range(count):
                self.fruits.append(spawn_item(bomb_chance))

        for f in self.fruits:
            if not f.sliced:
                f.update()
                if f.is_out():
                    if f.kind != "bomb":
                        self.lives -= 1
                        if self.lives <= 0:
                            self.game_over = True

        self.fruits = [f for f in self.fruits if not f.sliced and not f.is_out()]

        for h in self.halves:
            h.update()
        self.halves = [h for h in self.halves if not h.is_out()]

    def draw(self, screen):
        self.graphics.draw_game(
            screen,
            self.fruits,
            self.halves,
            self.score,
            self.lives,
            self.blade_trail,
            self.is_mouse_down,
            self.game_over
        )
        pygame.display.flip()
```

File: helper/physics.py

```
import random
import math
from helper.coords import SCREEN_WIDTH, SCREEN_HEIGHT, GRAVITY, FRUIT_SIZE

class Fruit:
    def __init__(self, kind, x, y, vx, vy, size=FRUIT_SIZE):
        self.kind = kind
        self.x = float(x)
        self.y = float(y)
        self.vx = float(vx)
        self.vy = float(vy)
        self.size = size
        self.sliced = False

    def update(self):
        self.vy += GRAVITY
        self.x += self.vx
        self.y += self.vy
```

```
    def is_out(self):
        return self.y > SCREEN_HEIGHT + 20 and self.vy > 0

    def is_hit(self, mouse_pos):
        center_x = self.x + self.size / 2
        center_y = self.y + self.size / 2
        radius = self.size / 2
        distance = math.hypot(mouse_pos[0] - center_x, mouse_pos[1] - center_y)
        return distance <= radius


class HalfPiece:
    def __init__(self, kind, is_left, x, y, vx, vy, size=FRUIT_SIZE):
        self.kind = kind
        self.is_left = is_left
        self.x = float(x)
        self.y = float(y)
        self.vx = float(vx)
        self.vy = float(vy)
        self.size = size

    def update(self):
        self.vy += GRAVITY
        self.x += self.vx
        self.y += self.vy

    def is_out(self):
        return self.y > SCREEN_HEIGHT + 20


def spawn_item(bomb_chance=0.2):
    if random.random() < bomb_chance:
        kind = "bomb"
    else:
        kind = random.choice(["apple", "mango", "strawberry"])

    x = random.randint(120, SCREEN_WIDTH - 180)
    y = SCREEN_HEIGHT
    vy = random.uniform(-15.0, -13.5)

    if x < SCREEN_WIDTH / 2:
        vx = random.uniform(0.8, 2.2)
    else:
        vx = random.uniform(-2.2, -0.8)

    return Fruit(kind, x, y, vx, vy)
```

Outputs

Figure 1: Initial Game Screen and Fruit Spawning:

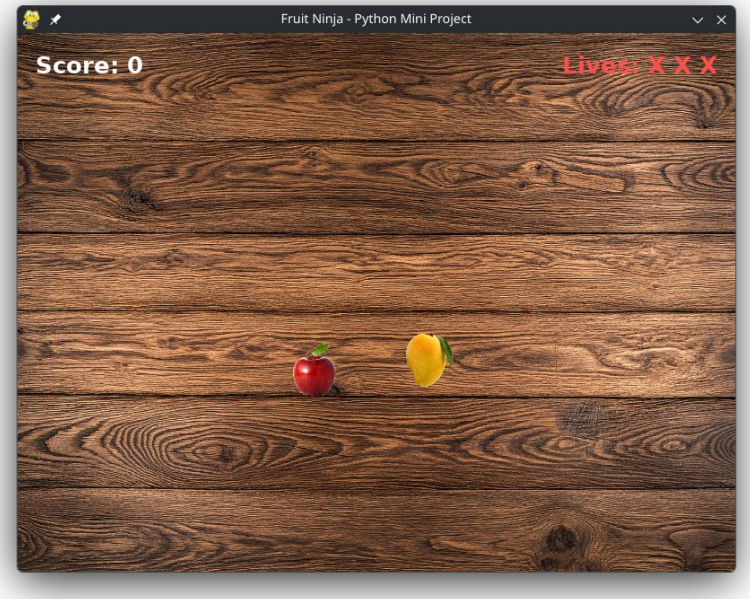


Figure 2: Active Gameplay and Fruit Slicing:



Figure 3: Game Over Screen and Restart Option:

